Software

Software is: (1) instructions (computer programs) that when executed provide desired features, function, and performance; (2) data structures that enable the programs to adequately manipulate information, and (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

Software characteristics

Software characteristics are classified into six major components.

1. **Functionality: Refers to the degree of performance of the software against its intended purpose.** In simple terms, this asks: does the software do what it's supposed to do? If you built an app to calculate taxes, functionality means it actually calculates taxes correctly and completely.
2. **Reliability: Refers to the ability of the software to provide desired functionality under the given conditions.** This is about consistency — can the software keep doing its job properly every time, under normal conditions, without crashing or giving wrong results? A reliable app doesn't fail randomly.
3. **Usability: Refers to the extent to which the software can be used with ease.** This is about how easy and comfortable the software is to use. Can a normal person figure out how to use it without a manual? Good usability means clear menus, simple instructions, and a smooth experience.
4. **Efficiency: Refers to the ability of the software to use system resources in the most effective and efficient manner.** This is about resources — memory, processing power, battery, storage. Efficient software gets the job done without wasting these resources. Think of two apps that do the same task: the one that uses less memory and runs faster is more efficient.
5. **Maintainability: Refers to the ease with which modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.** This is about how easy it is to fix or improve the software later. If a bug shows up, or the client wants a new feature, can a developer make that change quickly and safely, without breaking other parts of the program? Well-organized code is easier to maintain.

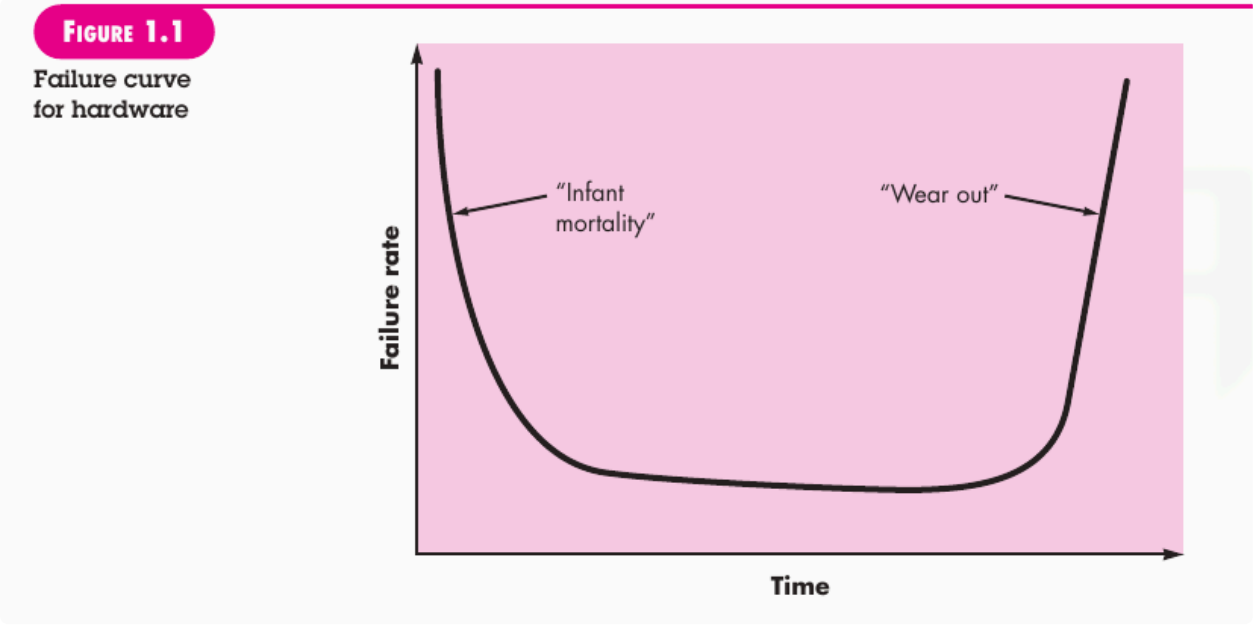
6. **Portability: Refers to the ease with which software developers can transfer software from one platform to another, without (or with minimum) changes.** In simple terms, it refers to the ability of software to function properly on different hardware and software platforms without making any changes to it. This is about flexibility across environments — for example, an app that works on both Windows and Mac without needing to be rewritten is considered portable.
   Robustness: Refers to the degree to which the software can keep functioning despite being provided with invalid data. This means the software can handle unexpected or bad input without completely breaking down. For example, if a user accidentally types letters into a field expecting numbers, robust software won't crash — it will handle the error gracefully, maybe by showing a warning instead.

Integrity: Refers to the degree to which unauthorized access to the software or data can be prevented. This is about security — how well the software protects itself and its data from people who aren't supposed to access it. High integrity means strong protection against hacking, unauthorized access, or data tampering.

Software vs Hardware characteristics

1. *Software is developed or engineered; it is not manufactured in the classical sense*
   In both software development and hardware manufacturing activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are nonexistent (or easily corrected) for software. Both activities require the construction of a “product,” but the approaches are different. Software costs are concentrated in engineering. This means that software projects cannot be managed as if they were manufacturing projects.

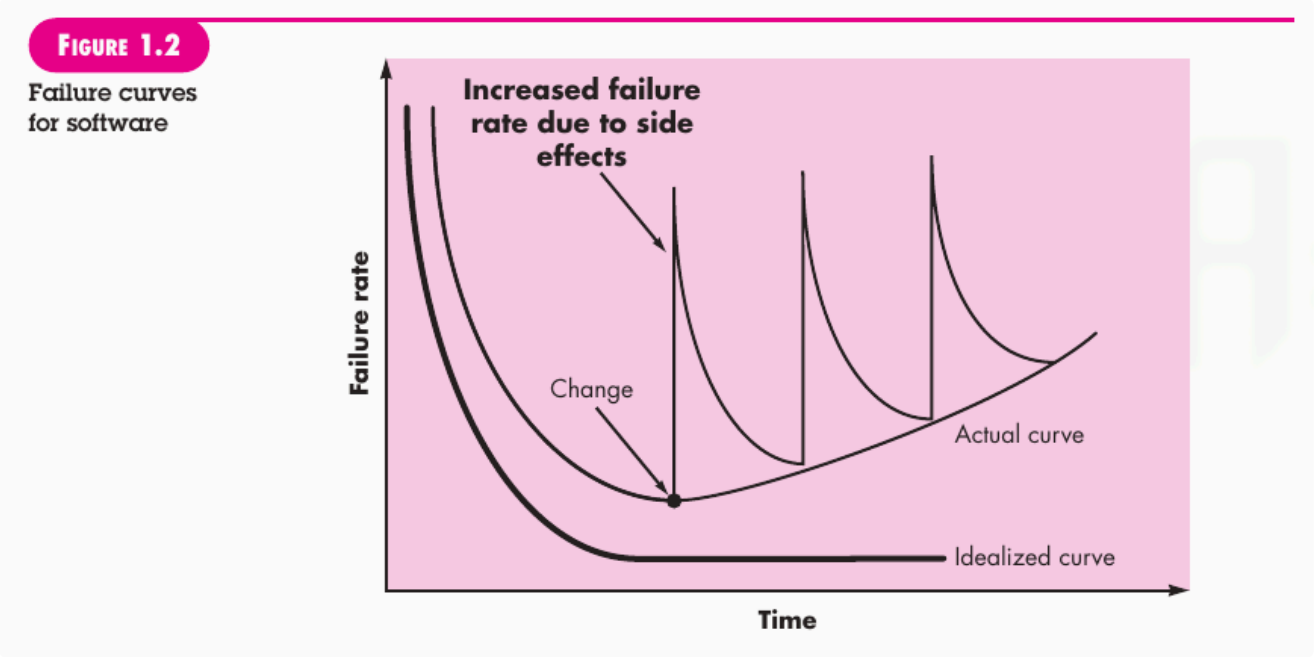
2. *Software doesn't "wear out."*



The "bathtub curve" describes how hardware failure rates change over time in a U-shaped pattern: high early on due to design or manufacturing defects, dropping to a low, steady rate once those defects are fixed, and rising again later as components wear out from cumulative wear, environmental stress, and age.

Unlike hardware, software doesn't wear out from environmental factors, so in theory its failure rate should follow an "idealized curve" — high initially due to undiscovered defects, then flattening out once those defects are fixed. But this idealized curve oversimplifies reality. In practice, software still deteriorates, just not through physical wear. Instead, each time the software is changed or updated, new errors are likely introduced, causing the failure rate to spike again before it can settle back down. Over repeated changes, the baseline failure rate gradually creeps upward — this is software deterioration caused by

change, not aging.



Software maintenance inherently more complex than hardware maintenance, since fixing it means addressing root design or implementation errors rather than simply replacing a worn component.

3. *Although the industry is moving toward component-based construction, most software continues to be custom built.*
   In the hardware world, component reuse is a natural part of the engineering process. The reusable components have been created so that the engineer can concentrate on the truly innovative elements of a design.
   A software component should be designed and implemented so that it can
   be reused in many different programs. Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the software engineer to create new applications from reusable parts.`

The evolving role of software

Software Evolution is a term which refers to the process of developing software initially, then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities etc. The evolution process includes fundamental activities of **change analysis, release planning**, system implementation and releasing a system to customers.

The necessity of Software evolution: Software evaluation is necessary just because of the following reasons:

1. **Change in requirement with time:** Over time, an organization's needs and way of working change. So the software they use also needs to change regularly to keep performance high.
2. **Environment change:** As the work environment changes, the tools used in it need to change too. The same applies to software — when the surrounding technology or industry changes, old software must be updated with new features to keep up.
3. **Errors and bugs:** As software gets older, it becomes less accurate and struggles to handle growing workloads efficiently. This makes it necessary to update or replace old software so it stays reliable and can handle current demands.
4. **Security risks:** Outdated software is more vulnerable to cyberattacks and can put sensitive data at risk. To avoid this, security patches and updates need to be checked regularly. If the software can't defend against current cyber threats, it needs to be updated.
5. **For having new functionality and features:** To keep performance high and processing fast, organizations need to keep improving their software over time. This ensures users and stakeholders can keep working efficiently.

Legacy software

Dayani-Fard and his colleagues [Day99] describe legacy software in the following way:

> *"Legacy software systems . . . were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms. The pro-liferation of such systems is causing headaches for large organizations who find them costly to maintain and risky to evolve."*

Liu and his colleagues [Liu98] extend this description by noting that

> *“many legacy systems remain supportive to core business functions and are ‘indispensable’ to the business.”*

Legacy systems sometimes have inextensible designs, convoluted code, poor or nonexistent documentation, test cases and results that were never archived, a poorly managed change history.

Legacy systems often evolve for one or more of the following reasons:

- The software must be adapted to meet the needs of new computing environments or technology.
- The software must be enhanced to implement new business requirements.
- The software must be extended to make it interoperable with other more modern systems or databases.
- The software must be re-architected to make it viable within a network environment.

Software Engineering

According to the definition proposed by Fritz Bauer at the seminal conference

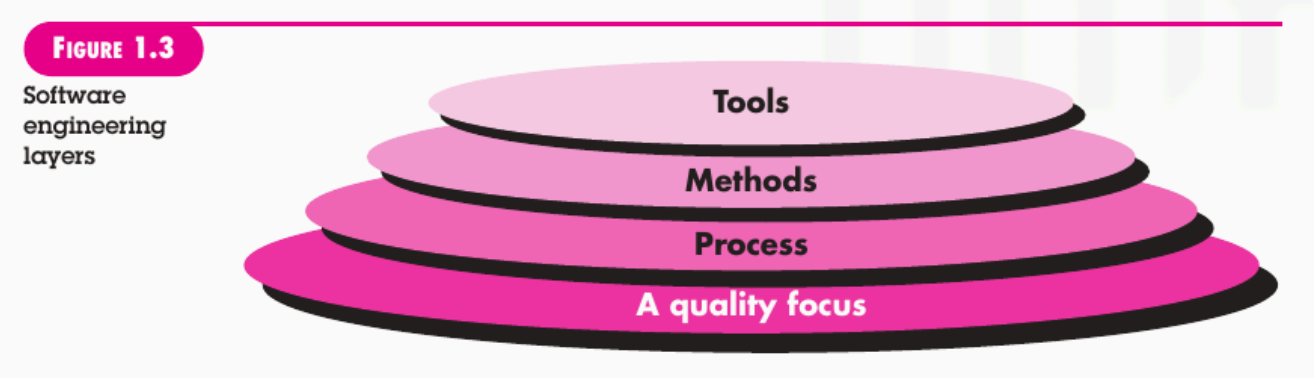
> "[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines."

The IEEE has developed a more comprehensive definition when it states:

> "Software Engineering: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1)."

A layered technology

Software engineering is a layered technology. The bedrock that supports software engineering is a quality focus.



Process layer

The foundation for software engineering is the process layer.
Process defines a framework that must be established for effective delivery of software engineering technology.
The software process forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.

Software engineering methods

Software engineering methods provide the technical how-to's for building software.
Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support.
Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive techniques.

Software engineering tools

Software engineering tools provide automated or semiautomated support for the process and the methods.
When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called *computer-aided software engineering*, is established.

The Software Process

A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.

- An activity strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
- An action (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model).
- A task focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

Process framework

A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.
A generic process framework for software engineering encompasses five activities:

1. **Communication**: Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders). The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.
2. **Planning**: A software project is a complicated journey, and the planning activity creates a "map" that helps guide the team as it makes the journey. The map—called *a software project plan*—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.
3. **Modeling**: A software engineer creates models to better understand software requirements and the design that will achieve those requirements.
4. **Construction**: This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.
5. **Deployment**: The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

For many software projects, framework activities are applied iteratively as a project progresses. That is, communication, planning, modeling, construction, and deployment are applied repeatedly through a number of project iterations. As each increment is produced, the software becomes more and more complete.

Software Life Cycle Models

Software life cycle

The term software life cycle has been defined to imply the different stages (or phases) over which a software evolves from an initial

customer request for it, to a fully developed software, and finally to a stage where it is no longer useful to any user, and then it is discarded.

The stage where the customer feels a need for the software and forms rough ideas about the required features is known as the ***inception stage.***
Starting with the inception stage, a software evolves through a series of identifiable stages (also called ***phases***) on account of the development activities carried out by the developers, until it is fully developed and is released to the customers.

When the users start to use the software, it signals the start of the operation aka ***maintenance phase***. The maintenance phase usually involves continually making changes to the software to accommodate the bug-fix and change requests from the user. Finally the software is retired.

"The life cycle of a software represents the series of identifiable stages through which it evolves during its life time."

Software Development Life Cycle (SDLC) model

A *software development life cycle (SDLC) model* (also called *software life cycle model* and *software development process model*) describes the different activities that need to be carried out for the software to evolve in its life cycle.

An SDLC graphically depicts the different phases through which a software evolves. It is usually accompanied by a textual description of the different activities that need to be carried out during each phase.

Importance of a life cycle model

The primary advantage is that it encourages development of software in a systematic and disciplined manner.

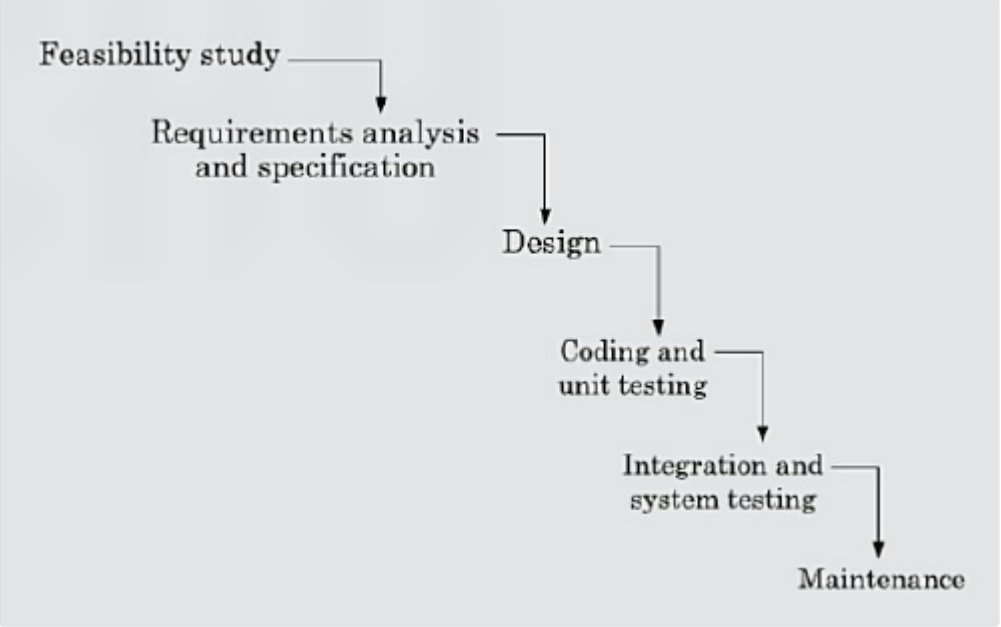
Adhering to a process is especially important to the development of professional software needing team effort.
When a software is developed by a team, it is necessary to have a precise understanding among the team members as to—when to do what. If not, each member at any time would do whatever activity he feels like doing. This would be an open invitation to developmental chaos and project failure.
For example, one member might start writing the code for his part while

making assumptions about the input results required from the other parts, another might decide to prepare the test documents first, and some other developer might start to carry out the design for the part assign to him. In this case, several problems can arise in interfacing the different parts and manage the overall development.
The use of a suitable life cycle model is crucial to the successful completion of a team-based development project.

Classic Waterfall model

Classical waterfall model is intuitively the most obvious way to develop software.
All other life cycle models can be thought of as being extensions of the classical waterfall model.

The classical waterfall model divides the life cycle into a set of phases.



- It can be easily observed from this figure that the diagrammatic representation of the classical waterfall model resembles a multi-level waterfall.

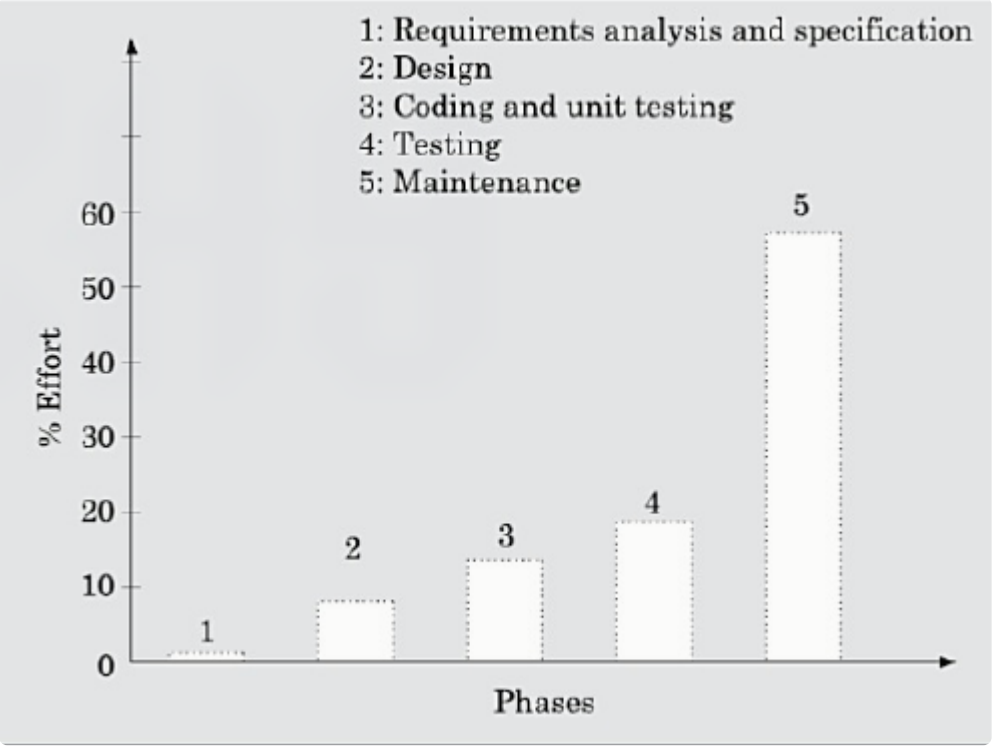
Phases of the classical waterfall model

The phases starting from the feasibility study to the integration and system testing phase are known as the ***development phases***.
At the completion of the development phases, the software is delivered to the customer.

After the delivery of software, customers start to use the software signalling the commencement of the ***operation phase***.

The last phase is also known as the ***maintenance phase*** of the life cycle.

In the waterfall model, different life cycle phases typically require relatively different amounts of efforts to be put in by the development team. Among all the life cycle phases, the maintenance phase normally requires the maximum effort.



However, among the development phases, the integration and system testing phase requires the maximum effort in a typical development project.

Feasibility study

The main focus of the feasibility study stage is to determine whether it would be **financially** and **technically** feasible to develop the software. The feasibility study involves carrying out several activities such as collection of basic information relating to the software such as

- The different data items that would be input to the system
- The processing required to be carried out on these data
- The output data required to be produced by the system
- As well as various constraints on the development
  These collected data are analysed to perform at the following:
- **Development of an overall understanding of the problem**: It is necessary to first develop an overall understanding of what the customer requires to be developed. For this, only the the important requirements of the customer need to be understood and the details

of various requirements such as the screen layouts required in the graphical user interface (GUI), specific formulas or algorithms required for producing the required results, and the databases schema to be used are ignored.

- **Formulation of the various possible strategies for solving the problem**: In this activity, various possible high-level solution schemes to the problem are determined.
- **Evaluation of the different solution strategies**: The different identified solution schemes are analysed to evaluate their benefits and shortcomings. Such evaluation often requires making approximate estimates of the resources required, cost of development, and development time required. If none of the solutions is feasible due to high cost, resource constraints, or some technical reasons, it would require the project to be abandoned.

Requirements analysis and specification

The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer and to document them properly.
This phase consists of two distinct activities, namely ***requirements gathering and analysis***, and ***requirements specification***.

Requirements gathering and analysis

The goal of the requirements gathering activity is to collect all relevant information regarding the software to be developed from the customer with a view to clearly understand the requirements. For this, first requirements are gathered from the customer and then the gathered requirements are analysed.
The goal of the requirements analysis activity is to weed out the **incompleteness** and **inconsistencies** in these gathered requirements.

Requirements specification

After the requirement gathering and analysis activities are complete, the identified requirements are documented. This is called a ***software requirements specification* (SRS) document**.
The SRS document normally serves as a contract between the development team and the customer. The SRS document is therefore an important document which must be thoroughly understood by the development team, and reviewed jointly with the customer.

Design

The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language.
In technical terms, during the design phase the *software architecture* is derived from the SRS document.

Two distinctly different design approaches are popularly being used at present—the ***procedural*** and ***object-oriented design*** approaches.

Procedural design approach

This traditional approach is currently widely used in software development projects.
It is based on data flow-oriented design approach.

It consists of two important activities - ***structured analysis*** and ***structured design***.
Structured analysis is where the detailed structure of the problem is examined. While structured design is where the results of former activity is transformed into the software design.
During structured analysis, the functional requirements specified in the SRS document is decomposed into sub-functions and the data flow between them is analysed and represented diagrammatically in the form of DFDs.
Structured design is performed after the structured analysis activity is completed. It consists of two main activities - ***architectural design***(aka *high-level design*) and ***detailed design*** (aka *low-level design*).

- High-level design involves decomposing the system into modules, representing the interfaces and the invocation relationships among the modules. It is sometimes referred to as *software architecture.*
- In Low-level design, internals of individual modules such as the data structures and algorithms are designed and documented.

Object-oriented design approach

In this approach, objects in the problem and solution domains and the relationships among them are identified. The object structure is further refined to obtain the detailed design.
This approach is credited to have several benefits such as lower development time and effort and better maintainability of the software.

Coding and unit testing

The purpose of the coding and unit testing phase is to translate a software design into source code and to ensure that individually each function is working correctly.
The coding phase is also sometimes called the ***implementation phase***.
Each component of the design is implemented as a program module. The end-product of this phase is a set of program modules that have been individually unit tested.
The specific activities carried out during unit testing include designing test cases, testing, debugging to fix problems, and management of test cases.

Integration and system testing

During the integration and system testing phase, various modules from the previous phase are integrated in a planned manner.
The module integration are normally carried out incrementally over a number of steps. During each integration step, previous planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules are integrated successfully and tested, the full working system is obtained.

System testing usually consists of three different kinds of testing activities:

- **Alpha-Testing**: System testing performed by the development team.
- **Beta-Testing**: System testing performed by a friendly set of customers.
- **Acceptance-Testing**: System testing performed after the software product is delivered, by customers to determine whether to accept the delivered software or to reject it.

Maintenance

The ratio of relative effort of developing a typical software product and the total effort spent on its maintenance is roughly 40:60.
Maintenance is required in the following three types of situations:

- **Corrective maintenance**: This type of maintenance is performed to correct errors that were not discovered during the product development phase.

- **Perfective maintenance**: It is carried out to improve the performance of the system or enhance the functionality of the system based on customer's requests.
- **Adaptive maintenance**: It is usually required for porting the software to work in a new environment. For example - to work on a new computing platform or with a new operating system.

Shortcomings of the classical waterfall model

- **No feedback paths**: Once a phase is complete, the activities carried out in it and any artifacts produced in this phase are considered to be final and are closed for any rework. This requires that all activities during a phase are flawlessly carried out. The classical waterfall model assumes that no error is ever committed by the developers during any of the life cycle phases, and therefore, incorporates no mechanism for error correction. While, in practical development environments, the developers do commit a large number of errors. Therefore, in any non-trivial software development project, it becomes nearly impossible to strictly follow the classical waterfall model of software development.
- **Difficult to accommodate change requests**: This model assumes that all customer requirements can be completely and correctly defined at the beginning of the project. But, it is hard to achieve this even in ideal project scenarios. The customers' requirements usually keep on changing with time.
- **Inefficient error corrections**: This model defers integration of code and testing tasks until it is very late when the problems are harder to resolve.
- **No overlapping of phases**: This model recommends that the phases be carried out sequentially—new phase can start only after the previous one completes. In a practical software development scenario, rather than having a precise point in time at which a phase transition occurs, the different phases need to overlap for cost and efficiency reasons.

Iterative Waterfall Model

The iterative waterfall model can be thought of as incorporating the necessary changes to the classical waterfall model to make it usable in

practical software development projects.

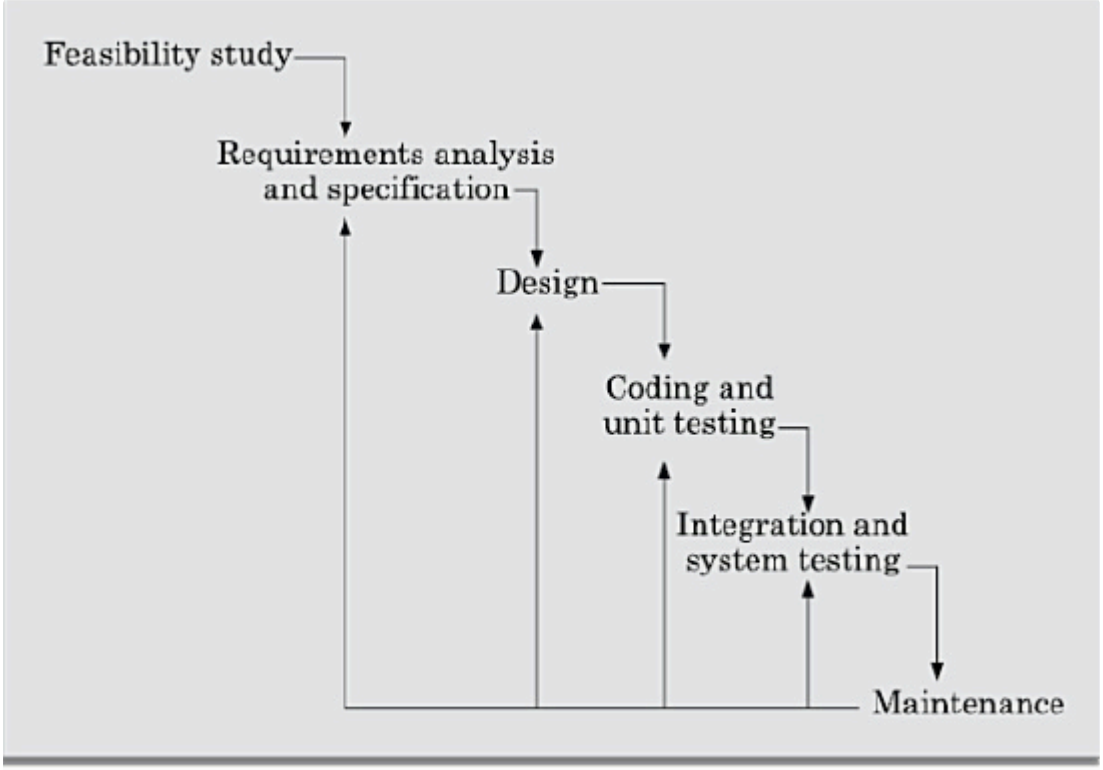


Figure 2.3: Iterative waterfall model.

The feedback paths allow for correcting errors committed by a programmer during some phase, as and when these are detected in a later phase. For example, if during the testing phase a design error is identified, then the feedback path allows the design to be reworked and the changes to be reflected in the design documents and all other subsequent documents.
There is no feedback path to the feasibility stage. This is because once a team having accepted to take up a project, does not give up the project easily due to legal and moral reasons.

Shortcomings of Iterative Waterfall Model

- **Difficult to accommodate change requests** – Requirements must be fixed before development starts, and detailed plans are built around them. Any change later on requires reworking both the plan and already-completed artifacts. Yet in practice, requirement changes are almost inevitable — customers often only fully understand their needs after seeing a working system.
- **No incremental delivery** – The entire software is built and tested before any delivery happens. Since this can take months or years, the customer's business needs may shift in the meantime, making the final product a poor fit by the time it's delivered.

- **No phase overlap** – The model expects each phase to fully finish before the next begins, which is unrealistic for most real projects and can create bottlenecks. In practice, phases are often forced to overlap to keep things moving.
- **Expensive error correction** – Testing and validation only happen after development is complete, so defects are caught late — leading to costly rework, delays, and increased costs.
- **Limited customer interaction** – Customers are typically involved only at the very start and end of the project, increasing the risk that the final product doesn't match their real needs.
- **Heavyweight documentation** – The model places heavy emphasis on documentation, which — while helpful for maintenance and review — consumes significant time and can reduce team efficiency.
- **Poor support for risk handling and code reuse** – The model isn't well-suited for projects that involve significant risk or that depend on reusing existing code/artifacts, which is now common in modern (especially customized/service-based) software projects.

Prototyping Model

This model suggests building a working prototype of the system, before development of the actual software.
A prototype is a toy and crude implementation of a system.
It has limited functional capabilities, low reliability, or inefficient performance as compared to the actual software.
A prototype can be built very quickly by using several shortcuts. The shortcuts usually involve developing inefficient, inaccurate, or dummy functions.
Normally the term *rapid prototyping* is used when software tools are used for prototype construction.

Advantages of Prototyping Model

- It is advantageous to use the prototyping model **for development of the graphical user interface (GUI)** part of an application. Through the use of a prototype, it becomes easier to illustrate the input data formats, messages, reports, and the interactive dialogs to the customer. This is a valuable mechanism for gaining better understanding of the customers' needs.
- The prototyping model is especially useful **when the exact technical solutions are unclear to the development team**. A prototype can help

them to critically examine the technical issues associated with product development. For example, consider a situation where the development team has to write a command language interpreter as part of a graphical user interface development. This risk can be resolved by developing a prototype compiler for a very small language to understand the issues associated with writing a compiler for a command language.

- An important reason for developing a prototype is that it is impossible to “get it right” the first time. The prototyping model can be deployed when development of highly optimised and efficient software is required.

Life cycle activities of prototyping model

In prototyping model, software is developed through two major activities —**prototype construction** and **iterative waterfall-based software**

development.

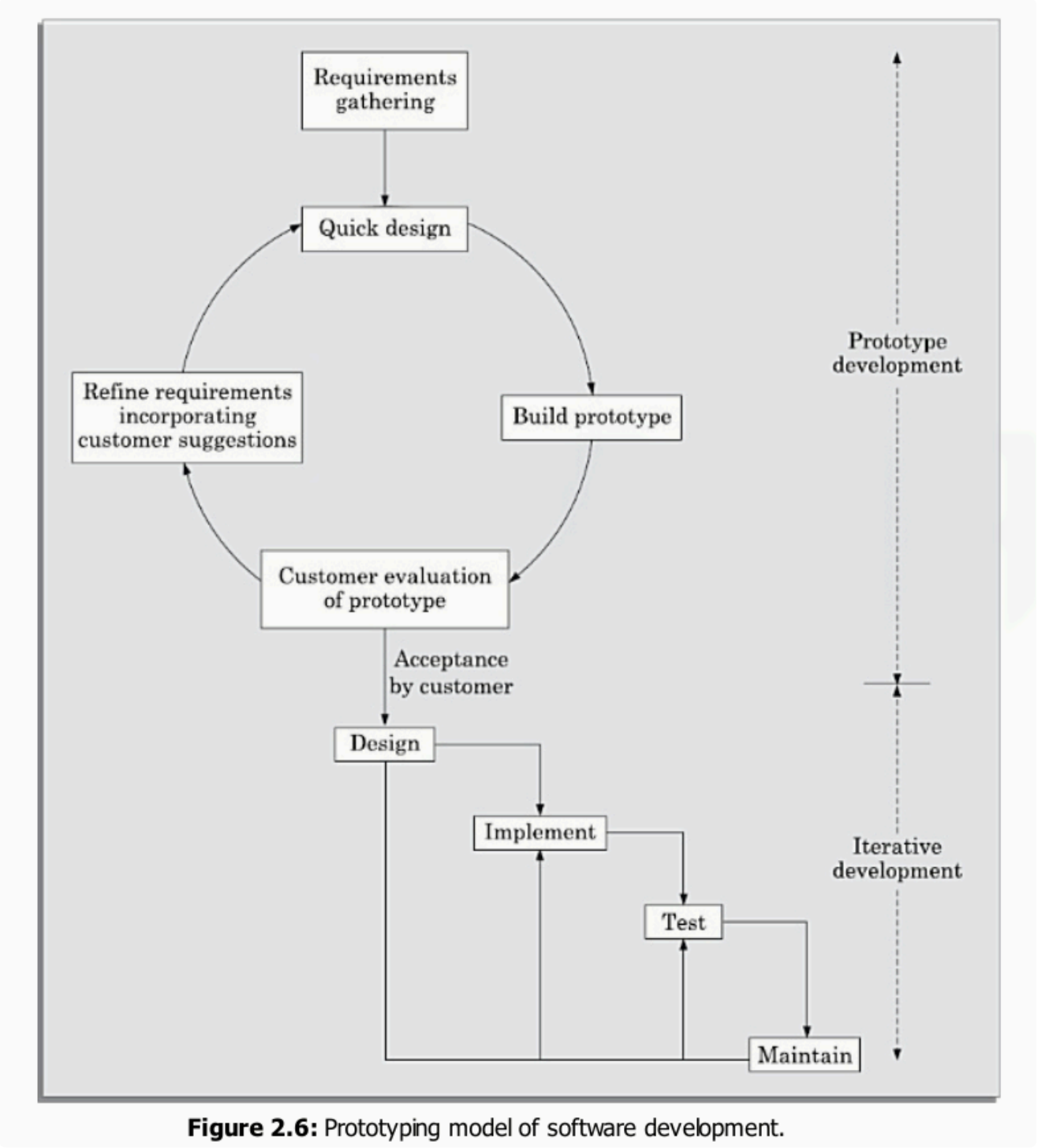


Figure 2.6: Prototyping model of software development.

1. **Prototype Development**: Prototype development starts with an initial requirements gathering phase. A quick design is carried out and a prototype is built. The developed prototype is submitted to the customer for evaluation. Based on the customer feedback, the requirements are refined and the prototype is suitably modified. This cycle of obtaining customer feedback and modifying the prototype continues till the customer approves the prototype.
2. **Iterative Development**: Once the customer approves the prototype, the actual software is developed using the iterative waterfall approach. In spite of the availability of a working prototype, the SRS

document is usually needed to be developed. However, for GUI parts, the requirements analysis and specification phase becomes redundant since the working prototype that has been approved by the customer serves as an animated requirements specification. The code for the prototype is usually thrown away. However, the experience gathered from developing the prototype helps a great deal in developing the actual system.

Weaknesses of the prototyping model

- The prototype model can increase the cost of development for projects that are routine development work and do not suffer from any significant risks.
- Even when a project is susceptible to risks, the prototyping model is effective only for those projects for which the risks can be identified upfront before the development starts. Since the prototype is constructed only at the start of the project, the prototyping model is ineffective for risks identified later during the development cycle. The prototyping model would not be appropriate for projects for which the risks can only be identified after the development is underway.

Evolutionary Model

In evolutionary model, the software is developed over a number of increments. At each increment, a concept (feature) is implemented and is deployed at the client site. The software is successively refined and feature-enriched until the full software is realised.

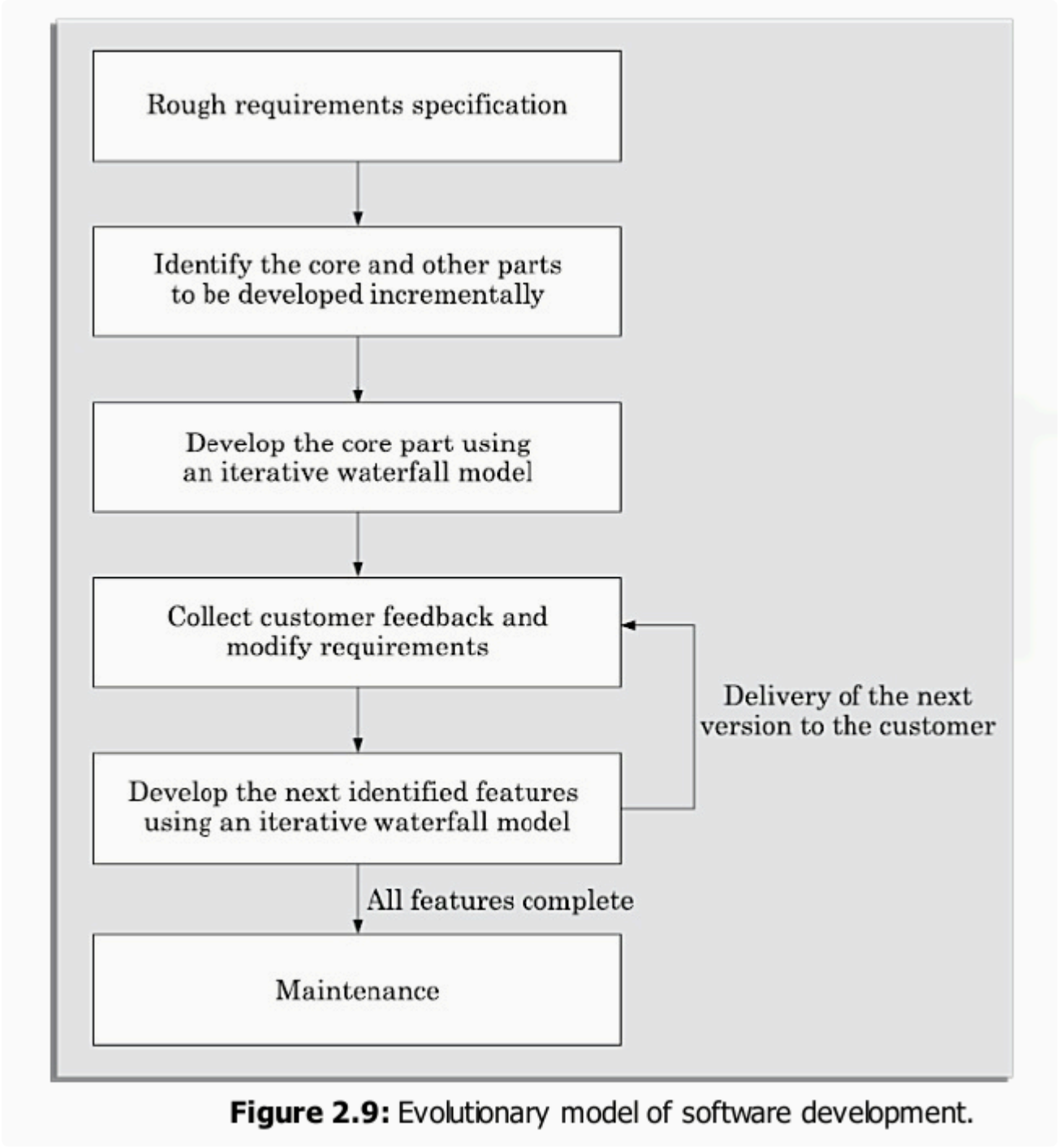


Figure 2.9: Evolutionary model of software development.

In the evolutionary model, the requirements, plan, estimates, and solution evolve over the iterations, rather than fully defined and frozen in a major up-front specification effort.
Such evolution is consistent with the pattern of unpredictable feature discovery and feature changes that take place in new product development.

The evolutionary software development process is sometimes referred to as *design a little, build a little, test a little, deploy a little model*. This means that after the requirements have been specified, the design, build, test, and deployment activities are iterated

Advantages

Two important advantages of using this model are the following:

- **Effective elicitation of actual customer requirements**: In this model, the user gets a chance to experiment with a partially developed software much before the complete requirements are developed. This elicits user requirements with the help of feedback obtained on the delivery of different versions of the software. As a result, the change requests after delivery of the complete software gets substantially reduced.
- **Easy handling change requests**: In this model, handling change requests is easier as no long term plans are made. Consequently, reworks required due to change requests are normally much smaller compared to the sequential models.

Disadvantages

- **Feature division into incremental parts can be non-trivial**: For many development projects, especially for small-sized projects, it is difficult to divide the required features into several parts that can be incrementally implemented and delivered. Further, even for larger problems, often the features are so interwined and dependent on each other.
- **Ad hoc design**: Since at a time design for only the current increment is done, the design can become ad hoc without specific attention being paid to maintainability and optimality.

Applicability

- The evolutionary model is normally useful for very large products, where it is easier to find modules for incremental implementation.
- Often evolutionary model is used when the customer prefers to receive the product in increments so that he can start using the different features as and when they are delivered.
- Another important category of projects for which the evolutionary model is suitable, is projects using object-oriented development since it is easy to partition the software into stand alone units in terms of the classes.

Spiral model

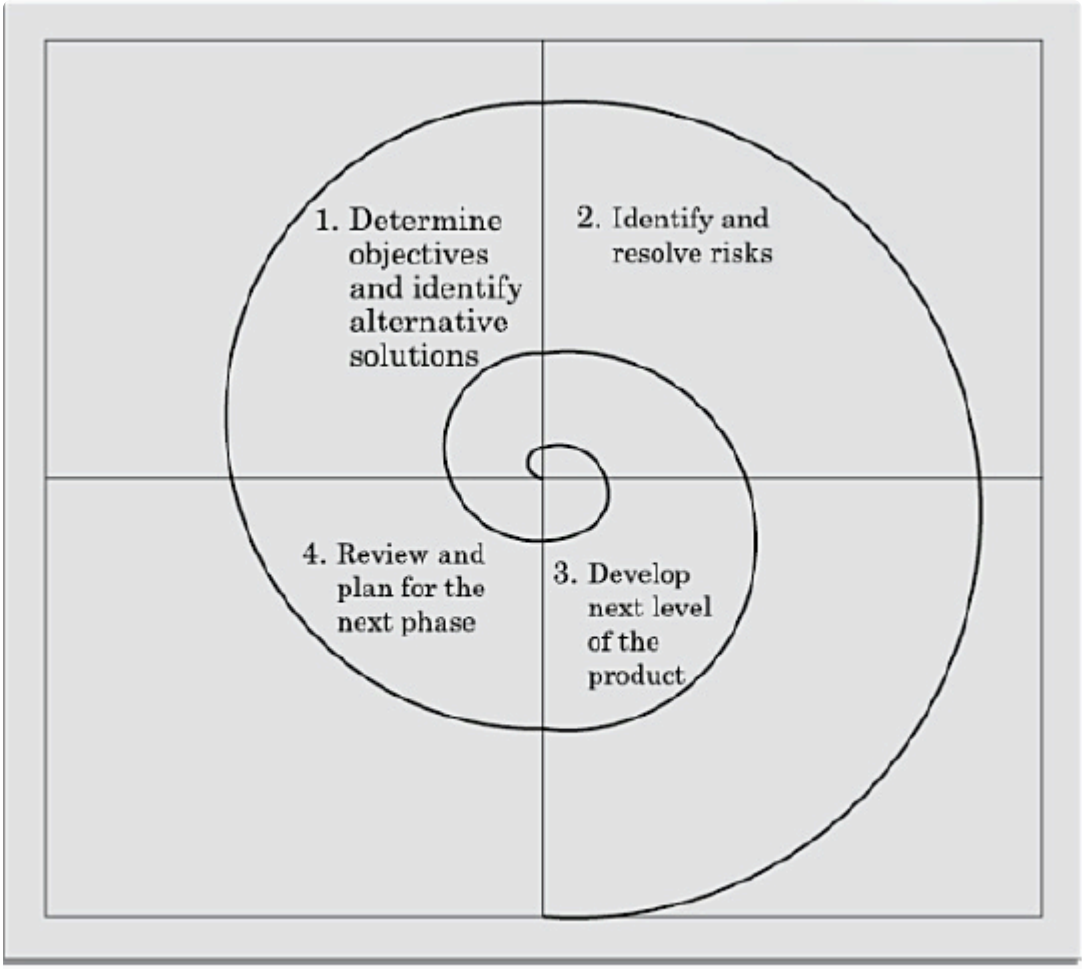


Figure 2.10: Spiral model of software development.

This model gets its name from the appearance of its diagrammatic representation. The exact number of loops of the spiral is not fixed and can vary from project to project.

Each loop of the spiral is called a ***phase*** of the software process. The exact number of phases through which the product is developed can be varied by the project manager depending upon the project risks.

A prominent feature of the spiral model is handling unforeseen risks that can show up much after the project has started.
In contrast to prototyping model that requires risks to be identified completely before the project starts, since prototype is constructed only at the start of the project, **spiral model requires prototypes to be built at the start of every phase**.
Over each loop, one or more features of the product are elaborated and analysed and the risks at that point of time are identified and are resolved through prototyping. Based on this, the identified features are implemented.

Risk handling in spiral model

As an example, consider a project for which a risk can be that data access from a remote database might be too slow to be acceptable by the customer. This risk can be resolved by building a prototype of the data access subsystem and experimenting with the exact access rate. If the data access rate is too slow, possibly a caching scheme can be implemented or a faster communication scheme can be deployed to overcome the slow data
access rate. Such risk resolutions are easier done by using a prototype as the pros and cons of an alternate solution scheme can evaluated faster and inexpensively, as compared to experimenting using the actual software application being developed.

Phases of the spiral model

Each phase in this model is split into four sectors (or quadrants).

- **Quadrant 1**: The objectives are investigated, elaborated, and analysed. Based on this, the risks involved in meeting the phase objectives are identified. Alternative solutions possible for the phase under consideration are proposed.
- **Quadrant 2**: During the second quadrant, the alternative solutions are evaluated to select the best possible solution. To be able to do this, the solutions are evaluated by developing an appropriate prototype.
- **Quadrant 3**: Activities during the third quadrant consist of developing and verifying the next level of the software. At the end of the third quadrant, the identified features have been implemented and the next version of the software is available.
- **Quadrant 4**: Activities during the fourth quadrant concern reviewing the results of the stages traversed so far (i.e. the developed version of the software) with the customer and planning the next iteration of the spiral.

The radius of the spiral at any point represents the cost incurred in the project so far, and the angular dimension represents the progress made so far in the current phase.

In this model, the project manager plays the crucial role of tuning the model to specific projects.

Advantages and Disadvantages of spiral model

Advantages

- For projects having many unknown risks that might show up as the development proceeds, the spiral model would be the most appropriate development model to follow.

Disadvantages

- To the developers of a project, the spiral model usually appears as a complex model to follow, since it is risk-driven and is more complicated phase structure. It would therefore be counterproductive to use, unless there are knowledgeable and experienced staff in the project.
- It is not very suitable for use in the development of outsourced projects, since the software risks need to be continually assessed as it is developed.

Agile Development Models

The agile software development model was proposed in the mid-1990s to overcome the serious shortcomings of the waterfall model of development.

The agile model was primarily designed to help a project to adapt to change requests quickly. Thus, **a major aim of the agile models is to facilitate quick project completion.**
Agility is achieved by fitting the process to the project, i.e. removing activities that may not be necessary for a specific project. Also, anything that that wastes time and effort is avoided.

Agile model is used as an umbrella term to refer to a group of development processes. These processes share certain common characteristics, but do have certain subtle differences among themselves.
a few popular agile sdlc models are the following:

- crystal
- atern (formerly dsdm)
- feature-driven development
- **scrum**
- extreme programming (xp)
- learn development
- unified process

In the agile model, the requirements are decomposed into many small parts that can be incrementally developed.
Each iteration is intended to be small and easily manageable and lasting for a couple of weeks only.
At a time, only one increment is planned, developed, and then deployed at the customer site.

The time to complete an iteration is called a ***time box***. The implication of the term time box is that the end date for an iteration does not change. The development team can, however, decide to reduce the delivered functionality during a time box if necessary.

For establishing close contact with the customer during development and to gain a clear understanding of the domain-specific issues, **each agile project usually includes a customer representative in the team**.
At the end of each iteration, stakeholders and the customer representative review the progress made and re-evaluate the requirements.

Agile model emphasise face-to-face communication over written documents.
It is recommended that the development team size be deliberately kept small (5–9 people).
If a large project is required to be developed using the agile model, it is likely that the collaborating teams might work at different locations. In this case, the different teams are needed to maintain as much daily contact as possible through various mediums.

Agile development projects usually deploy ***pair programming.***
In pair programming, two programmers work together at one work station. One types in code while the other reviews the code as it is typed in. The two programmers switch their roles every hour or so.

Scrum

In the scrum model, a project is divided into small parts of work that can be incrementally developed and delivered over time boxes that are called ***sprints***. The software therefore gets developed over a series of manageable chunks.

Each sprint typically takes only a couple of weeks to complete. At the end of each sprint, stakeholders and team members meet to assess the progress made and the stakeholders suggest to the development team

any changes needed to features that have already
been developed and any overall improvements that they might feel necessary.

In the scrum model, the team members assume three fundamental roles —

- software owner
- scrum master, and
- team member.
  The software owner is responsible for communicating the customers vision of the software to the development team.
  The scrum master acts as a liaison between the software owner and the team, thereby facilitating the development work.